c:\hpc-23-feb-2026\.\37\file_creator.c

```
#include<stdio.h>
#include<string.h>
int main()
{
char buffer[1024];
FILE *f;
int x;
memset(buffer,'A',sizeof(buffer));
f=fopen("test.data","wb");
for(x=1;x<500*1024;++x)
{
fwrite(buffer,sizeof(buffer),1,f);
}
fclose(f);
return 0;
}
```

Thinking Machines Classroom Codes

c:\hpc-23-feb-2026\.\37\serial_reader.cpp

```
#include<iostream>
#include<stdio.h>
#include<chrono>
using namespace std;
int main()
{
char *buffer;
buffer=new char[8*1024];
FILE *f;
int read_count;
auto start=chrono::high_resolution_clock::now();
f=fopen("test.data","rb");
while(1)
{
read_count=fread(buffer,1,sizeof(buffer),f);
if(read_count==0) break;
}
fclose(f);
auto end=chrono::high_resolution_clock::now();
auto d=chrono::duration_cast<chrono::milliseconds>(end-start);
cout<<d.count()<<" milliseconds"<<endl;
return 0;
}
```

Thinking Machines Classroom Codes

c:\hpc-23-feb-2026\.\37\serial_preader.cpp

```
#include<iostream>
#include<stdio.h>
#include<chrono>
#include<fcntl.h>
#include<unistd.h>
using namespace std;
int main()
{
char *buffer;
//buffer=new char[8*1024];
buffer=new char[4*1024*1024];
int file_descriptor;
int read_count;
auto start=chrono::high_resolution_clock::now();
file_descriptor=open("test.data",O_RDONLY);
int offset=0;
while(1)
{
read_count=pread(file_descriptor,buffer,4*1024*1024,offset);
if(read_count<=0) break;
offset+=read_count;
}
close(file_descriptor);
auto end=chrono::high_resolution_clock::now();
auto d=chrono::duration_cast<chrono::milliseconds>(end-start);
cout<<d.count()<<" milliseconds"<<endl;
delete [] buffer;
return 0;
}
```